

Opis pliku w katalogu plików FAT32

- ◆ we wczesnych wersjach systemu Windows (i w systemie DOS) nazwy plików mogły zawierać co najwyżej 11 znaków, w tym 3 znaki przeznaczone były na rozszerzenie nazwy pliku; później wprowadzono możliwość kodowania nazw plików o długości do 255 znaków, przy zachowaniu dotychczasowej struktury katalogu, co spowodowało, że nazwa pliku musi być kodowana na kilku pozycjach w katalogu; nazwy 11-znakowe lub krótsze kodowane są w dotychczasowy sposób;

Nazwa pola	Offset	Rozmiar
Nazwa pliku (krótka)	0	11
Atrybuty pliku	11	1
bajt zarezerwowany	12	1
pole dodatkowe precyzujące czas utworzenia pliku (następne pole) w postaci liczby jednostek 10 ms	13	1
Czas utworzenia pliku	14	2
Data utworzenia pliku	16	2
Data ostatniego dostępu do pliku	18	2
Nr początkowej pozycji w tablicy FAT32 (dwa starsze bajty)	20	2
Czas ostatniej modyfikacji pliku	22	2
Data ostatniej modyfikacji pliku	24	2
Nr początkowej pozycji w tablicy FAT32 (dwa młodsze bajty)	26	2
Rozmiar pliku	28	4

- ◆ w pierwszym bajcie pola nazwy pliku mogą także występować kody specjalne:

- 00H — pozycja dotychczas w ogóle nie używana;
- E5H — opis pliku skasowanego (kasowanie pliku polega na wpisaniu kodu E5H do pierwszego bajtu);
- 05H — pierwszym znakiem nazwy pliku jest E5H (znak E5H jest dopuszczalnym znakiem w zestawach narodowych)
- 2EH — opis podkatalogu; jeśli następny bajt zawiera także kod 2EH (tj. '..'), to analizowana pozycja opisuje katalog nadrzędny w stosunku do bieżącego — w tym przypadku pole 'Nr początkowej pozycji ...' wskazuje na numer pozycji w FAT, zawierającej katalog nadrzędny; zero oznacza, że katalogiem nadrzędnym jest katalog główny;

♦ bajt atrybutu określa własności pliku lub podkatalogu:

- 01H tylko do odczytu
- 02H plik ukryty
- 04H plik systemowy
- 08H etykieta dysku
- 10H katalog
- 20H znacznik archiwizacji

wartość 0FH oznacza, że pozycja katalogu zawiera część (dłuższą) nazwy pliku;

♦ data i czas kodowane są wg poniższych formatów:

15	11	10	5	4	0
godziny		minuty		sekundy / 2	

♦ w podanym formacie liczba sekund podana jest na 5 bitach z dokładnością do 2; wyjątkowo, czas utworzenia pliku kodowany jest za pomocą 3 bajtów, z których pierwszy określa liczbę jednostek 10-milisekundowych ($0 \div 199$), a dwa pozostałe kodowane są w podany sposób;

15	9	8	5	4	0
rok – 1980		miesiąc		dzień	

- ♦ pozycje opisujące długie nazwy plików (ang. LFN – long file name) przechowywane są w dodatkowych rekordach 32-bajtowych przed rekordem opisującym plik — w tym przypadku rekord opisujący plik zawiera nazwę pliku obciętą do sześciu pierwszych znaków, po których następuje znak tyldy ~ i kolejny numer porządkowy (w razie wystąpienia krótkiej tej samej nazwy pliku);
- ♦ charakterystycznym elementem rekordów dodatkowych jest kod 0FH umieszczony w polu atrybutu — identyfikuje on rekord zawierający długą nazwę pliku (lub jej część); ponieważ właściwe informacje o pliku przechowywane są w rekordzie podstawowym, więc w każdym rekordzie dodatkowym można umieścić 26 bajtów nazwy pliku — przy nazwach dłuższych stosowany jest łańcuch rekordów;
- ♦ nazwa kodowana jest w 16-bitowym kodzie Unicode; często jest to znak ASCII, po którym występuje bajt zerowy.

Nazwa pola	Offset	Rozmiar
Nr rekordu (w sekwencji rekordów zawierających nazwę)	0	1
Pole nazwy pliku (5 znaków Unicode)	1	10
Atrybuty pliku — zawsze kod 0FH	11	1
(pole zarezerwowane)	12	1
Suma kontrolna rekordu podstawowego *)	13	1
Pole nazwy pliku (6 znaków Unicode)	14	12
Nr początk. pozycji w tablicy FAT (= 0000H)	26	2
Pole nazwy pliku (2 znaki Unicode)	28	4

*) sumę kontrolną rekordu podstawowego (tj. rekordu zawierającego nazwę krótką i inne informacje o pliku), wyznacza się w celu sprawdzenia czy bieżący rekord dodatkowy jest skojarzony z rekordem podstawowym, umieszczonym dalej w katalogu, bezpośrednio za rekordami dodatkowymi; wg opisu R. Brown'a suma kontrolna obliczana jest poprzez sumowanie wszystkich znaków nazwy krótkiej, przy czym przed dodaniem kolejnego znaku, dotychczas uzyskana wartość jest przesuwana cyklicznie o jedną pozycję w prawo;

- ♦ długa nazwa pliku zakończona jest liczbą 0 zapisaną na 16 bitach; zakończenie to jest pomijane, jeśli ostatni znak długiej nazwy pliku znajduje się na końcu rekordu (offset 30); ewentualnie niewykorzystane pola nazwy pliku wypełnia się kodami FFFFH.

Struktura tablicy rozmieszczenia plików FAT

- ◆ ze względu na znaczną liczbę sektorów na dysku twardym, w celu ułatwienia zarządzania plikami, system operacyjny posługuje się jednostkami zawierającymi kilka sektorów — jednostka taka nazywana jest *klasterem* (*wiązką*) lub jednostką alokacji pliku (ang. cluster); zazwyczaj klaster zawiera 1, 2, 4 lub więcej sektorów;
- ◆ tablica rozmieszczenia plików FAT (ang. file allocation table) zawiera informacje o stanie wszystkich klasterów obszaru danych dysku; kolejne wiązki numerowane są: 2, 3, ...;
- ◆ struktura informacji w tablicy FAT:
 - pierwsze dwie pozycje są rezerwowane dla specjalnych informacji;
 - trzecia pozycja tablicy FAT zawiera informacje o pierwszym klasterze obszaru danych (tj. klasterze oznaczonym numerem 2);
 - następne pozycje tablicy FAT zawierają informacje o kolejnych klasterach;
 - każda pozycja w tablicy FAT używana przez plik wskazuje następną pozycję w FAT dla tego pliku — sekwencja numerów kolejnych pozycji FAT jest taka sama jak numery wiązek składających się na plik, a zatem opisane jest położenie pliku na dysku;
- ◆ stosuje się trzy metody kodowania elementów tablicy: 12-bitową dla dyskietek (omawianą w podanym dalej przykładzie) i 32-bitową dla dysków twardych (FAT32); kodowanie 16-bitowe nie jest obecnie używane;
- ◆ rozmiar klastera zależy zwykle od rozmiaru dysku; w systemie Windows XP dla FAT32 stosuje się klasterzy 4 kB dla dysków < 8 GB, 8 kB dla dysków < 1 GB i 16 kB dla dysków < 32 GB;

- ♦ kody stosowane w tablicach FAT podaje poniższa tablica (wartości szesnastkowe):

Kodowanie 12-bitowe	Kodowanie 16-bitowe	Kodowanie 32-bitowe	
000	0000	00000000	wiązka dostępna (niezajęta)
FF7	FFF7	FFFFFFFF7	wiązka uszkodzona (tj. zawierająca błędny sektor)
FF8 ÷ FFF	FFF8 ÷ FFFF	0FFFFFFF8 ÷ 0FFFFFFFFF	kody z tego zakresu wskazują, że wiazka zawiera końcową część pliku
002 ÷ FF6	0002 ÷ FFF6	00000002 ÷ 0FFFFFFF6	wskaźnik na następny element

- ♦ przy przyjętej organizacji pliki nie muszą być umieszczone sekwencyjnie w kolejnych wiazkach; jeśli plik zostaje rozszerzony (np. w trakcie edycji), to system operacyjny poszukuje pierwszej wolnej wiazki i dołącza ją do sekwencji numerów.

Przykład: poniżej podano przykładową analizę struktur systemowych dyskietki o pojemności 1,44 MB — dyskietka zawiera 2880 sektorów 512-bajtowych. Na dyskietce zapisano kolejno cztery pliki, a później plik OS3.C poddano edycji, w wyniku której objętość pliku zwiększyła się do 3231 bajtów.

Zawartość katalogu głównego (przed edycją pliku). Kolejne wiersze (z wyjątkiem pierwszego: etykieta dysku) opisują poszczególne pliki — w prawej kolumnie podana jest początkowa pozycja w tablicy FAT.

ANALIZA	FAT	0	12.96.16	14:11 pm	0	Vol
VGAPL	PAS	4561	10.91.14	8:36 am	2	
OS3	C	2339	12.96.16	13:49 pm	11	
CONFIG	SYS	200	12.96.16	13:51 pm	16	
DMA	WPG	3387	1.96.21	16:26 pm	17	

Początkowa część tablicy FAT (stan przed edycją pliku). Tłustym drukiem zaznaczono pozycje, które zostały zmienione po edycji pliku OS3.C.

	(2)	3	4	5	6	7	8
(8) 9	10	EOF	12	13	14	15	EOF
(16) EOF	18	19	20	21	22	23	EOF
(24) 0	0	0	0	0	0	0	0
(32) 0	0	0	0	BAD	BAD	BAD	0

Uwaga: pozycje z lewej strony tablicy FAT mają indeksy: 2 (niepełny wiersz), 8, 16, 24,; przykładowo liczba 9 na początku drugiego wiersza znajduje się na pozycji 8 tablicy FAT i wskazuje, że dalsza część pliku umieszczona jest w wiązce nr 9.

Zawartość katalogu po edycji pliku OS3.C

ANALIZA	FAT	0	12.96.16	14:11 pm	0	Vol
VGAPL	PAS	4561	10.91.14	8:36 am	2	
OS3	C	3231	12.96.16	14:30 pm	11	
CONFIG	SYS	200	12.96.16	13:51 pm	16	
DMA	WPG	3387	1.96.21	16:26 pm	17	

Początkowa część tablicy FAT po edycji pliku OS3.C

			3	4	5	6	7	8
(8)	9	10	EOF	12	13	14	15	24
(16)	EOF	18	19	20	21	22	23	EOF
(24)	25	EOF	0	0	0	0	0	0
(32)	0	0	0	0	BAD	BAD	BAD	0

Na podstawie katalogu i analizy tablicy FAT można określić położenie poszczególnych plików na dysku:

VGAPL.PAS	wiązki 2 — 10
CONFIG.SYS	wiązka 16
DMA.WPG	wiązki 17 — 23

OS3.C

przed edycją	wiązki 11 — 15
po edycji	wiązki 11 — 15, 24 — 25

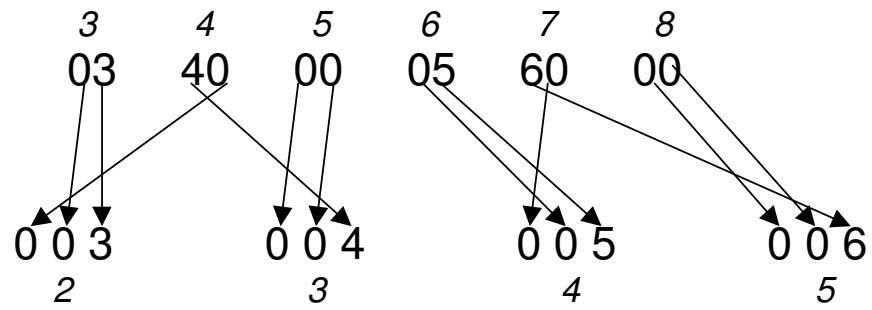
Kodowanie 12-bitowe w tablicy FAT

- ♦ poniżej podano fragment analizowanej poprzednio tablicy FAT (po edycji) w postaci zakodowanej; w tym systemie kodowania każda pozycja tablicy zapisana jest na 12-bitach w dość zawiły sposób;

00000000:	F0	FF	FF	03	40	00	05	60
00000008:	00	07	80	00	09	A0	00	FF
00000010:	CF	00	0D	E0	00	0F	80	01
00000018:	FF	2F	01	13	40	01	15	60
00000020:	01	17	F0	FF	19	F0	FF	00
00000028:	00	00	00	00	00	00	00	00

- ♦ w celu odczytania wskazanej pozycji tablicy postępuje się następująco:
 - nr potrzebnej pozycji należy pomnożyć przez 1.5 — część całkowita uzyskanego iloczynu określa kolejny numer bajtu od początku tablicy FAT;
 - należy odczytać z tablicy wskazany bajt i następny, tworząc z nich słowo 16-bitowe (wg konwencjonalnych zasad);
 - jeśli nr pozycji był parzysty, to należy wziąć 12 mniej znaczących bitów słowa, jeśli nieparzysty, to 12 bardziej znaczących bitów — uzyskana liczba stanowi zawartość analizowanej pozycji.
- ♦ podany algorytm ilustruje rysunek; jeśli chcemy odczytać pozycję nr 2, to odczytujemy dwa kolejne bajty zaczynając od bajtu o numerze 3 (część całkowita z iloczynu $2 \cdot 1.5$) — odczytaliśmy 03H 40H; następnie z obu bajtów tworzymy liczbę 16-bitową wg konwencji mniejsze niż (ang. little endian); z liczby 4003H bierzemy (nr pozycji parzysty) 12 najmniej znaczących bitów — otrzymujemy 003H, co stanowi zawartość pozycji nr 2 analizowanej tablicy FAT12.

Fragment tablicy FAT12 w postaci zakodowanej



Ten sam fragment po rozkodowaniu

Dostęp do plików w systemie Unix (Linux)

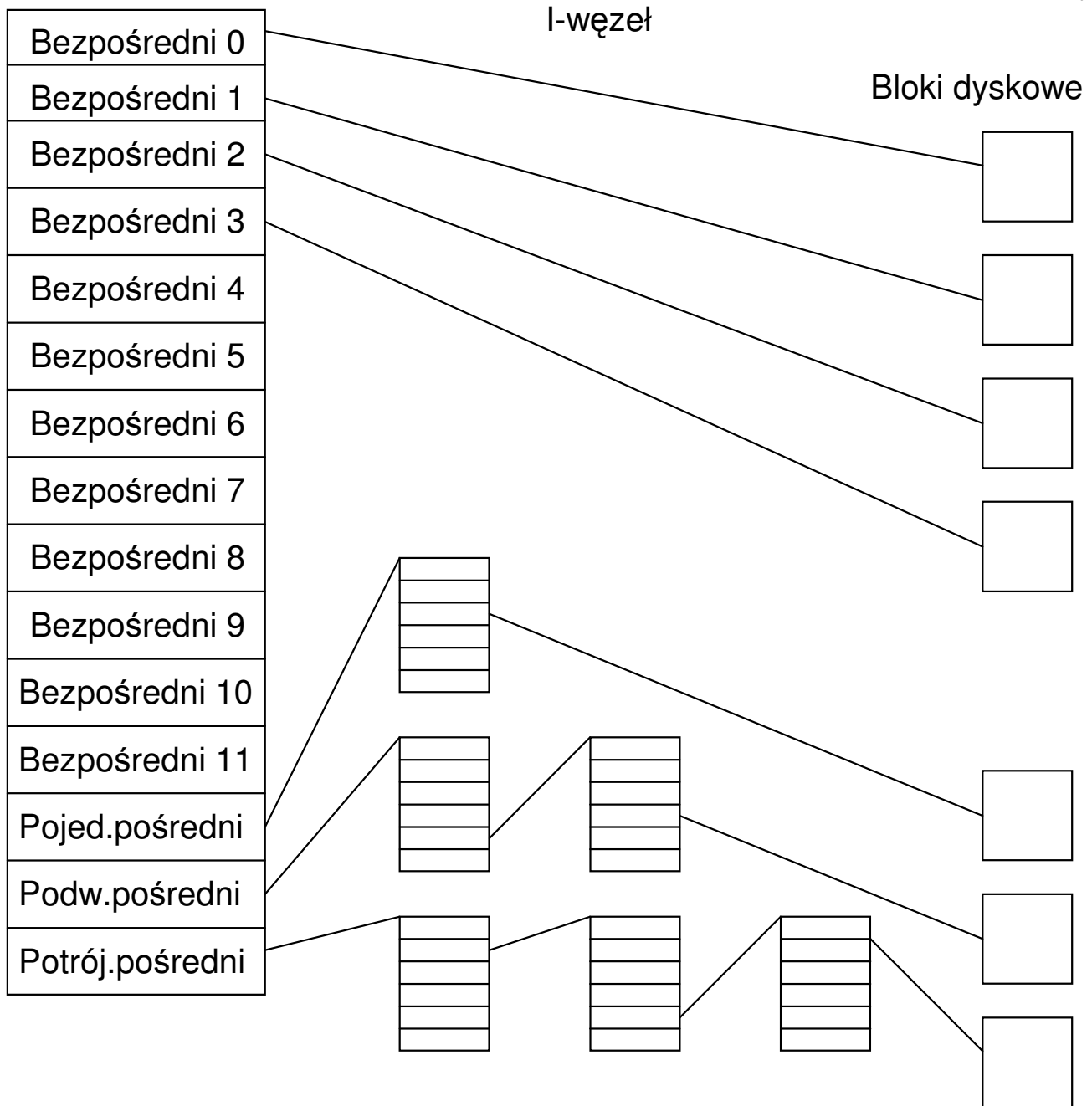
- ◆ w systemie Unix (Linux) podstawowe informacje o pliku przechowywane są w strukturze zwanej *i-węzłem* (ang. i-node); i-węzły przechowywane są w *tablicy i-węzłów*; każdemu otwartemu plikowi przyporządkowany jest dokładnie jeden i-węzeł; każdy i-węzeł zawiera:
 - opis położenia pliku na dysku i rozmiar pliku;
 - identyfikator właściciela pliku;
 - typ pliku (pliki zwykłe, katalogi, . . .);
 - prawa dostępu do pliku (właściciela pliku, grupowego właściciela pliku, pozostałych użytkowników);
 - czasy dostępu do pliku (czas ostatniej modyfikacji, czas ostatniego dostępu, czas ostatniej modyfikacji i-węzła pliku);
 - liczba dowiązań do pliku;
 - liczba procesów, które otworzyły ten plik;
 - inne;można zauważyć, że i-węzeł nie zawiera nazwy pliku;
- ◆ zwykle i-węzły przechowywane są w dużej tablicy na początku partycji dyskowej (lub też są rozproszone wewnątrz partycji) — w miarę otwierania plików są odczytywane i umieszczane w tablicy i-węzłów, która jest przechowywana w pamięci operacyjnej; można powiedzieć, że większość informacji zawartych w i-węźle (w tablicy) została odczytana z odpowiedniego i-węzła na dysku ;
- ◆ system plików odwołuje się do i-węzłów poprzez podanie indeksu w tablicy i-węzłów;
- ◆ specjalną grupę plików stanowią katalogi (ang. directories) — na dysku są one przechowywane tak jak zwykłe pliki, m.in. z każdym katalogiem związany jest i-węzeł; z kolei dane zapisane w katalogu zawierają nazwy plików wraz z indeksami przyporządkowanych im i-węzłów;

- ◆ taka organizacja plików umożliwia przyporządkowanie każdemu plikowi więcej niż jednej nazwy — w katalogach plików mogą być umieszczone różne nazwy wskazujące na ten sam i-węzeł; każdą nazwę określa się jako *dowiązanie* (ang. link);
- ◆ stosunkowo łatwe jest także przeniesienie pliku do innego katalogu: w katalogu trzeba utworzyć nazwę pliku wraz indeksem do tablicy i-węzłów i jednocześnie usunąć nazwę w dotychczasowym katalogu.

Przechowywanie plików na dysku w systemie Unix (Linux)

- ◆ system Unix występuje w wielu wersjach i odmianach — stosowane w nich systemy plików charakteryzują się znaczną różnorodnością w zakresie konstrukcji wewnętrznej; istotnym czynnikiem determinującym przyjęte rozwiązania techniczne jest zapewnienie wysokiego poziomu efektywności i niezawodności systemu, który pracuje w trybie wielodostępnym i wielozadaniowym;
- ◆ aktualnie najbardziej rozpowszechniony jest system plikowy UFS (ang. Unix file system); w dystrybucjach Linuxa używany jest system plikowy Ext2/Ext3;
- ◆ pamięć na dysku podzielona jest na bloki, które są adresowane za pomocą numerów; rozmiar bloku wynosi zazwyczaj $1 \div 8$ KB; dla uproszczenia dalszych rozważań pominiemy grupowanie bloków;
- ◆ pliki przechowywane są na dysku w postaci rozproszonej;
- ◆ i-węzeł zawiera tablicę adresów, za pomocą której można zlokalizować części pliku na dysku;
- ◆ proste odwzorowanie elementów tablicy adresów w bloki dyskowe powodowałoby konieczność tworzenia tablic o znacznych rozmiarach w przypadku dużych plików; ponadto byłoby korzystnie, gdyby rozmiar i-węzła nie zależał od rozmiaru pliku;
- ◆ biorąc pod uwagę podane postulaty w systemie Unix przyjęto, że tablica adresów bloków (oznaczona *i_block*) zawiera ustaloną liczbę pozycji, np. 15 w systemie Ext2, a każda pozycja zajmuje 4 bajty;

- ◆ początkowe 12 pozycji omawianej tablicy zawiera adresy bezpośrednie, tj. wskazujące konkretne bloki — dzięki temu dostęp do plików o niewielkich rozmiarach odbywa się bardzo szybko; dalsze pozycje zawierają numery bloków: *pojedynczego pośredniego* (ang. indirect block), *podwójnego pośredniego* (ang. double indirect block), *potrójnego pośredniego* (ang. tripple indirect block);
- ◆ dostęp do danych przez blok pośredni wymaga wczytania tego bloku, i znalezienia odpowiedniej pozycji z adresem bloku bezpośredniego;



- ♦ jeśli przyjąć, że w systemie stosowane są 4-kilobajtowe bloki dyskowe, a numer bloku zapisywany jest na 32 bitach, to maksymalny rozmiar pliku przekracza 4 TB:

12 bloków bezpośrednich po 4 KB = 48 KB

1 blok pośredni i 1024 bloki bezpośrednie = 4 MB

1 blok podwójnie pośredni i 1024 pośrednie = 4 GB

1 blok potrójnie pośredni i 1024 bloki podwójnie pośrednie = 4 TB

- ◆ procesy sięgają do danych w pliku podając położenie bajtu; przykładowo jeśli proces odwołuje się do bajtu 30000, to $30000/4096 = 7$ (reszta 1328), a zatem potrzebny bajt znajduje się w bloku, którego numer podany jest na pozycji 7 tablicy adresów bloków; wewnątrz bloku żądany bajt jest odległy o 1328 bajtów od początku bloku;
- ◆ bardziej skomplikowane jest odwołanie do bajtów, które nie są dostępne poprzez bloki bezpośrednio; przykładowo dostęp do bajtu 5 000 000 wymaga użycia bloku podwójnie pośredniego; pierwszy bajt dostępny poprzez blok podwójnie pośredni ma numer $48 \text{ KB} + 4 \text{ MB} = 49152 + 4\,194\,304 = 4\,243\,456$; bajt o numerze 5 000 000 jest więc bajtem o numerze 756 544 względem początku bloku podwójnie pośredniego; każdy blok pojedynczo pośredni daje dostęp do 4 MB, więc bajt o numerze 5 000 000 będzie się wskazywany pośrednio przez pozycję zerową bloku podwójnie pośredniego — po odczytaniu wyznaczonego w ten sposób bloku pośredniego poszukuje się w nim pozycji o indeksie $756\,544 / 4096 = 184$ (reszta 2880); zawartość pozycji o indeksie 184 wskazuje na blok dyskowy zawierający żądany bajt — jest on odległy 2880 bajtów od początku bloku.

Podstawowe koncepcje systemu plików NTFS

- ◆ system NTFS (ang. new technology file system) został opracowany pod koniec lat osiemdziesiątych ubiegłego stulecia (udostępniony w systemie Windows NT w roku 1993); w intencji twórców miał stanowić wydajny i bezpieczny system plików, posiadający możliwości rozbudowy; w szczególności system może obsługiwać pliki o wielkich rozmiarach, obsługuje pliki w środowiskach wielonarodowych;
- ◆ system NTFS dziedziczy wiele cech systemu HPFS (ang. high performance file system), który został opracowany przez firmę IBM dla systemu operacyjnego OS/2; wersja systemu NTFS używana w systemie Windows XP numer 5.1 (używane jest oznaczenie 3.1);
- ◆ system NTFS jest w pewnym stopniu podobny do FAT, m.in. opisy plików znajdują się w pojedynczej (dalej omawianej) *nadrzędnej tablicy plików* (Master File Table); jednak, w przeciwieństwie do FAT, NTFS pozwala na ograniczenie dostępu do plików i katalogów dla poszczególnych użytkowników;
- ◆ poniżej wymieniono główne własności systemu NTFS:
 - możliwość określania praw dostępu do plików i katalogów,
 - prowadzenie dziennika transakcji, co pozwala na odtworzenie stanu systemu w przypadku uszkodzeń;
 - możliwość szyfrowania danych,
 - możliwość kompresji danych,
 - możliwość przydzielania przestrzeni na dysku indywidualnie dla każdego użytkownika;
 - możliwość obsługi bardzo dużych partycji (2^{64} bajtów) i plików (16 Terabajtów);
- ◆ ze względu na złożone struktury wewnętrzne, NTFS nie nadaje się do dysków o małej pojemności lub dyskietek; w takich przypadkach stosuje się system FAT;

Struktura partycji NTFS

- ◆ system NTFS tworzy woluminy, będące logicznymi partycjami, które obejmują część lub całość dysku twardego;
- ◆ w partycji NTFS wyróżnia się cztery obszary:
 - sektor ładujący,
 - nadrzędna tablica plików (ang. Master File Table),
 - pliki systemowe,
 - obszar danych;
- ◆ sektor ładujący ma strukturę podobną do używanej w FAT32: na początku sektora znajduje się trzybajtowy rozkaz skoku prowadzący do kodu uruchamiającego system operacyjny; dalej następują obszary OEM ID, BPB i EBPB; obszar OEM ID zawiera znaki „NTFS”, które identyfikują system plików;
- ◆ podobnie jak w innych systemach plików, tworzone są wiązki sektorów dyskowych, zwane *klastrami*; typowy rozmiar klastra dla woluminów o pojemności do 2 TB wynosi 4 kiB, jedynie dla woluminów o pojemności mniejszej od 2 GiB stosuje się mniejsze klastry;
- ◆ system NTFS posługuje się 64-bitowymi numerami klastrów nazywanymi LCN — logical cluster number — logiczny numer klastra; w szczególności klaster LCN 0 zawiera sektor ładujący; biorąc pod uwagę rozmiar klastra 4 kiB można zauważyć, że na poziomie współczesnej techniki rozmiar partycji jest praktycznie nieograniczony.

Nadrzędna tablica plików — Master File Table (MFT)

- ♦ w systemie NTFS wszelkie informacje przechowywane są w plikach; najważniejszym z plików systemowych jest *nadrzędna tablica plików* (ang. Master File Table – MFT) — zawiera ona informacje o położeniu wszystkich plików i katalogów na dysku; MFT można opisać jako tablicę, w której każdy wiersz przeznaczony jest na plik lub katalog; początkowe wiersze opisują pliki systemowe (metapliki), włączając w to samą tablicę MFT; struktura MFT pokazana jest na poniższym rysunku;

Plik 0	MFT
Plik 1	Fragment MFT (kopia)
Plik 2	Pliki systemowe (NTFS metadata files)
Plik 16	Pliki i katalogi użytkownika

- ♦ każdy plik opisany jest w tablicy MFT w postaci *rekordu plikowego* (1024 ÷ 4096 bajtów); rekord plikowy składa się z nagłówka oraz sekwencji występujących po sobie atrybutów w postaci pary (atrybut, wartość); struktura rekordu plikowego pokazana jest na rysunku;

- ♦ charakterystycznym elementem *nagłówka rekordu plikowego* są znaki „FILE” (w kodzie ASCII) umieszczone na początku nagłówka; nagłówek zawiera kilkanaście pól o różnym przeznaczeniu, m.in. podany jest faktyczny rozmiar rekordu plikowego (4-bajtowe pole o offsecie 18H) oraz zaalokowany rozmiar rekordu plikowego (4-bajtowe pole o offsecie 1CH) — w omawianym dalej przykładzie rekordu plikowego pola te zawierają liczby 168H i 400H; tak więc przykładowy rekord plikowy zajmuje 1024 bajty;

nagłówek
atrybut
atrybut
atrybut
znacznik końca rekordu plikowego:FFFFFFFFH

Atrybuty plików i katalogów

- ♦ w rekordzie plikowym, bezpośrednio za nagłówkiem, występuje sekwencja atrybutów w postaci pary (atrybut, wartość); atrybuty zawierają różne informacje o pliku, a przypadku małych plików (do 1 kiB) wartością jednego z atrybutów jest zawartość pliku;
- ♦ zatem pliki i katalogi, o rozmiarze nie przekraczającym 1 kiB (czasami 1500 bajtów), zapisywane są bezpośrednio w MFT; rozmiary te zależą od wielkości klastra i wersji systemu plików NTFS; w podanym dalej przykładzie rekord plikowy składa się z trzech atrybutów:

Informacja standardowa (atrybut typu 10H)	Nazwa pliku (katalogu) (atrybut typu 30H)	Zawartość pliku (atrybut typu 80H)
--	--	---------------------------------------

- ♦ każdy atrybut składa się z nagłówka i właściwych danych (nazywanych również strumieniem); wyróżnia się cztery formaty nagłówka atrybutu, m.in. nagłówek atrybutu rezydentnego bez nazwy zawiera 24 bajty; we wszystkich formatach dwa pierwsze pola mają jednakowe znaczenie:
 - typ atrybutu — 4-bajtowe pole o offsecie 00H (np. 00000030H oznacza atrybut nazwy pliku);
 - długość (obejmująca nagłówek i dane) — 4-bajtowe pole o offsecie 04H;
- ♦ każdy atrybut jest identyfikowany przez kod, a opcjonalnie także przez nazwę;
- ♦ *atrybut informacji standardowych* (kod 10H) opisuje prawa do zapisu i odczytu, zawiera cztery pola zawierające zakodowany czas oraz inne informacje;

- ◆ standardowo czas zapisywany jest w postaci liczby 64-bitowej reprezentującej liczbę jednostek 100 ns, które upłynęły od 1 stycznia 1601 r. do chwili wykonania podanej operacji; atrybut informacji standardowych zawiera m.in. cztery pola 64-bitowe:
 - czas utworzenia pliku (offset 00H),
 - czas ostatniej modyfikacji pliku (danych) (offset 08H),
 - czas zmiany rekordu w MFT (offset 10H),
 - czas ostatniego dostępu (offset 18H);

- ◆ *atrybut nazwy pliku* (kod 30H) zawiera nazwę pliku (offset 1AH), cztery pola określające czas, faktyczny i zaalokowany rozmiar pliku (pola o offsetach 30H i 28H), znaczniki (offset 38H: tylko do odczytu, ukryty, systemowy, skompresowany, zaszyfrowany, itp.) oraz inne informacje; nazwa pliku kodowana jest w standardzie Unicode, przy czym ilość znaków określa 1-bajtowe pole o offsecie 40H;

- ◆ *atrybut danych* (kod 80H) zawiera różnego rodzaju dane, a w szczególności w przypadku małych plików (o rozmiarze do 1 kiB) może to być zawartość pliku; w przypadku większych plików podane są wskaźniki na dane;

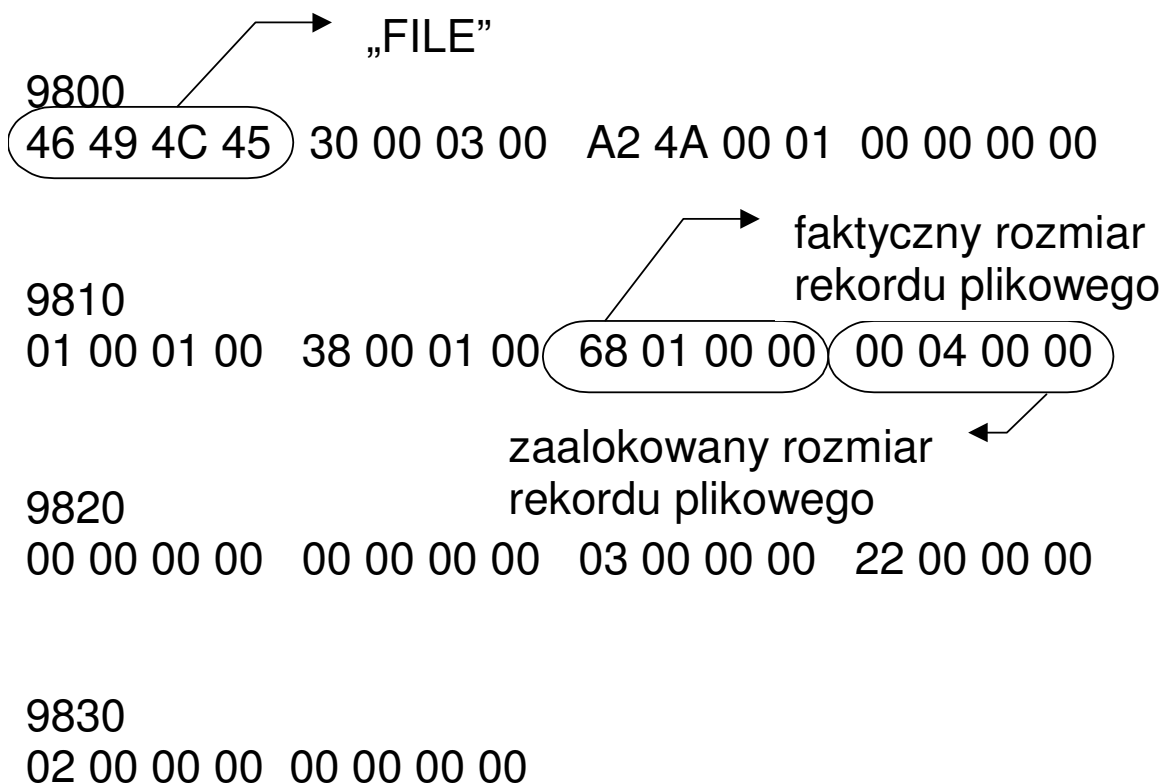
- ◆ dość często występuje także *atrybut praw dostępu* (kod 50H), w którym zawarta jest lista użytkowników właścicieli pliku i użytkowników z dostępem do pliku, wraz z typem dostępu;

Przykład kodowania rekordu plikowego

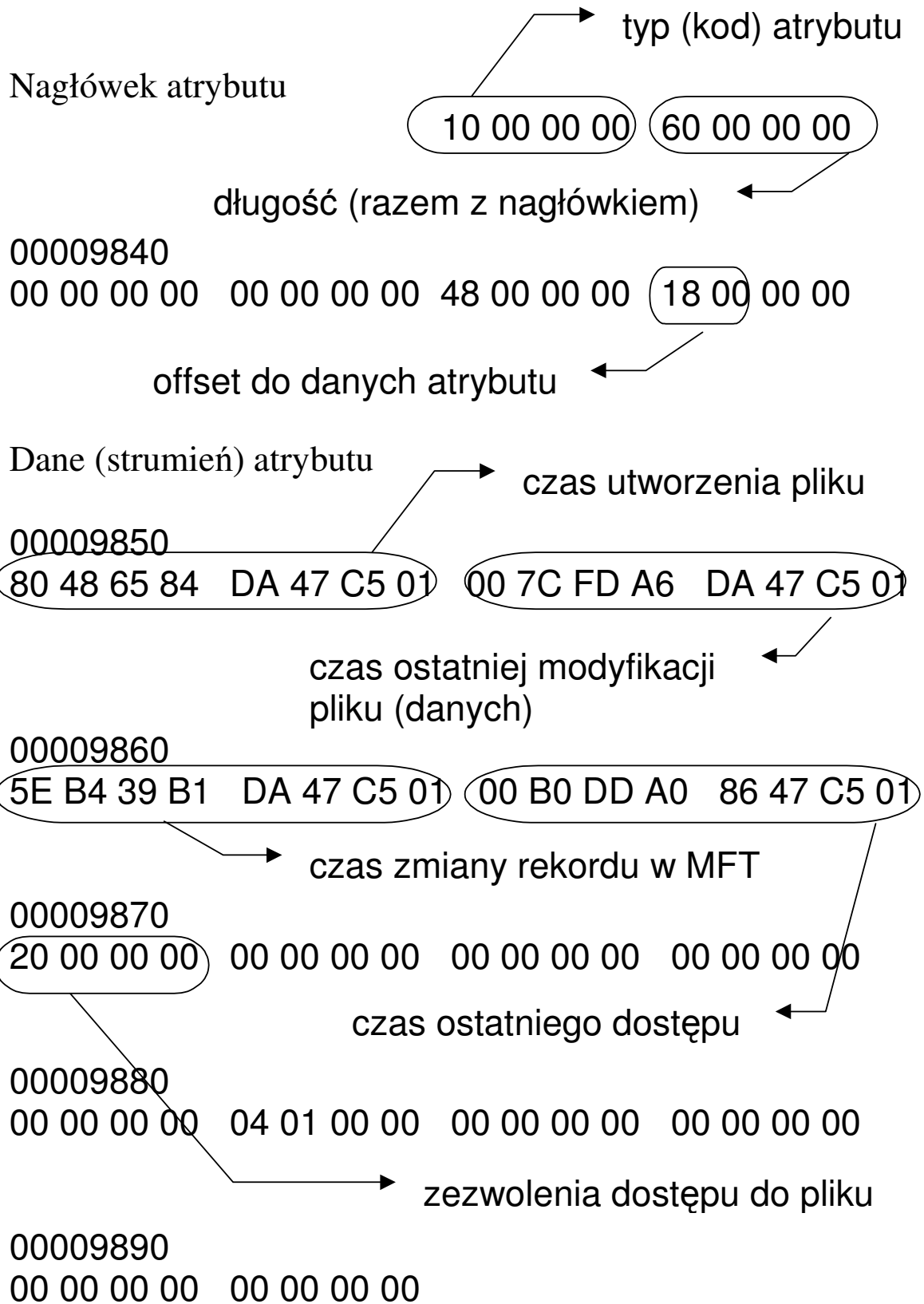
- ♦ poniżej podano przykładowy rekord plikowy opisujący plik „znaki.txt” (plik utworzono 23.04.2005 o godz. 10:01:28); rekord został odczytany z tablicy MFT; plik zawiera wielkie i małe litery alfabetu łacińskiego (58 bajtów) w postaci:

ABCDEFGHIJKLMNOPQRSTUVWXYZ
zyxwvutsrqponmlkjihgfedcba

Nagłówek rekordu plikowego

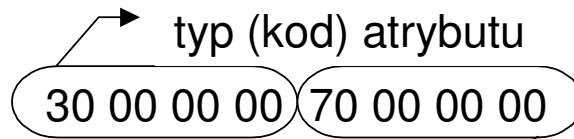


Atrybut "Informacje standardowe"



Atrybut "Nazwa pliku"

Nagłówek atrybutu



długość (razem z nagłówkiem)

offset do danych atrybutu
000098A0
00 00 00 00 00 00 02 00 54 00 00 00 18 00 01 00

Dane (strumień) atrybutu

000098B0
05 00 00 00 00 00 05 00 5E B4 39 B1 DA 47 C5 01

czas utworzenia pliku

000098C0
5E B4 39 B1 DA 47 C5 01 5E B4 39 B1 DA 47 C5 01

czas ostatniej modyfikacji pliku (danych)

000098D0
5E B4 39 B1 DA 47 C5 01 00 00 00 00 00 00 00 00

czas zmiany rekordu w MFT

000098E0
00 00 00 00 00 00 00 00 20 20 00 00 00 00 00 00

czas

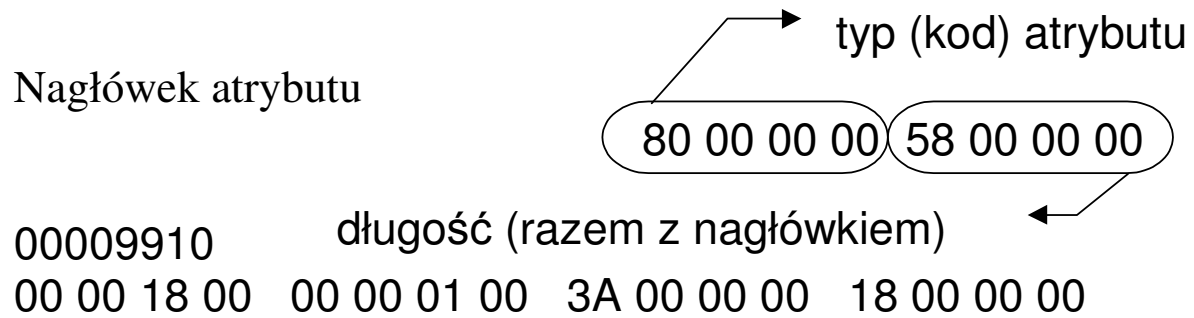
000098F0
09 03 7A 00 6E 00 61 00 6B 00 69 00 2E 00 74 00

liczba znaków nazwy pliku

00009900
78 00 74 00 00 00 00 00

„znaki.txt”

Atrybut "DATA"



Dane (strumień) atrybutu (58 bajtów stanowiących zawartość pliku)

00009920
41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50

00009930
51 52 53 54 55 56 57 58 59 5A 0D 0A 7A 79 78 77

00009940
76 75 74 73 72 71 70 6F 6E 6D 6C 6B 6A 69 68 67

00009950
66 65 64 63 62 61 0D 0A 0D 0A 00 00 00 00 00 00

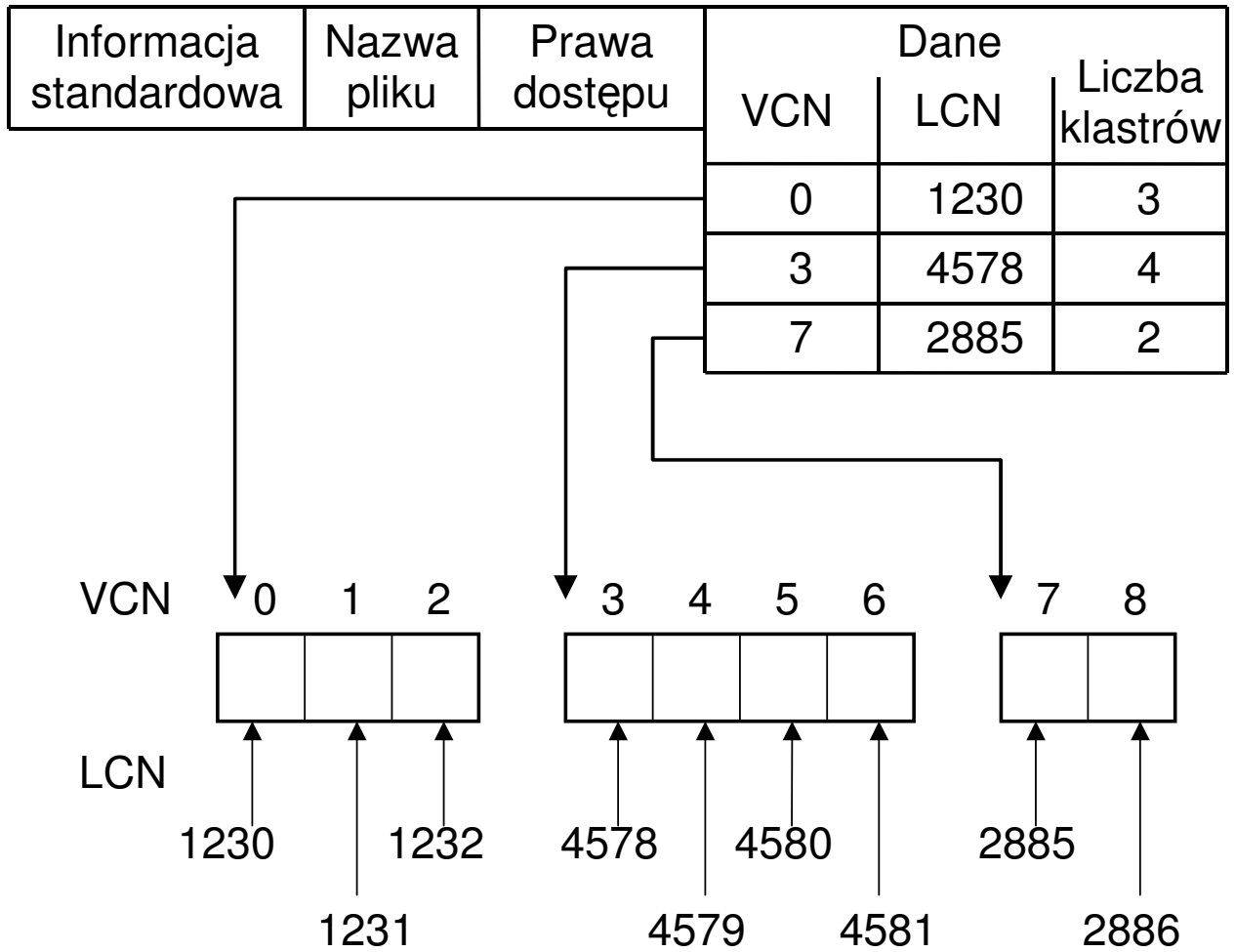
- ◆ na końcu rekordu plikowego znajduje się znacznik końca:
FF FF FF FF

- ♦ poniższa tablica zawiera zestawienie wybranych atrybutów używanych w systemie NTFS

Typ atrybutu	Nazwa	Nr	Opis
Informacje standardowe	\$STANDARD_INFORMATION	10H	zawiera znaczniki czasu, liczba łączy, prawa dostępu.
Nazwa pliku	\$FILE_NAME	30H	zawiera nazwę pliku lub katalogu
Informacje o woluminie	\$VOLUME_INFORMATION	70H	nazwa (wcześniej: wersja) woluminu
Dane	\$DATA	80H	Dane pliku
Indeks główny	\$INDEX_ROOT	90H	Używany do implementowania katalogów i indeksów
Mapa bitowa	\$BITMAP	B0H	dane tego atrybutu zawierają pole bitowe

Atrybuty rezydentne i nierezydentne

- ◆ w pliku zawarte są atrybuty różnej długości, np. atrybut nazwy pliku jest zwykle bardzo krótki, natomiast atrybut danych może być bardzo duży; w takich przypadkach niemożliwe jest przechowywanie wszystkich atrybutów pliku w jednym rekordzie plikowym;
- ◆ w tej sytuacji atrybuty dzieli się na dwie klasy:
 - rezydentne, których rozmiar nie przekracza wielkości rekordu plikowego, np. nazwa pliku oraz znaczniki czasu są zawsze rezydentne;
 - nierezydentne, które nie mieszczą się w jednym rekordzie i muszą być przechowywane poza tablicą MFT; w tym przypadku w miejscu, gdzie powinny się znajdować dane atrybutu, umieszczony jest ciąg wskaźników do poszczególnych części danych zwany *listą odwołań* (ang. runlist), która informuje o umiejscowieniu danych na dysku;
- ◆ zatem jeśli plik jest większy i nie mieści się w MFT, położenie zawartości pliku określa się za pomocą listy klastrów, w których zawarte są dane pliku; inne elementy są takie same jak dla małych plików;
- ◆ poniższy rysunek pokazuje przykładową zawartość wiersza tablicy MFT, opisującego plik zapisany na 9 klastrach;



VCN (virtual cluster number) — numer klastra wirtualnego
 LCN (logical cluster number) — numer klastra logicznego

- ◆ w przypadku gdy wszystkie atrybuty pliku będą nierezydentne, a łączna długość ich list odwołań będzie przekraczała wielkość jednego rekordu plikowego (gdy plik jest silnie sfragmentowany), plik otrzymuje nowy rekord plikowy zwany *rozszerzonym rekordem plikowym* (ang. extension file record), w którym lokowane są niemieszczące się atrybuty — od tej chwili oryginalny rekord nadany plikowi przy jego utworzeniu nosi nazwę *bazowego rekordu plikowego* (ang. base file record);
- ◆ bazowy rekord plikowy zawiera listę 64-bitowych wskaźników do pozostałych rekordów opisujących plik; każdy wskaźnik składa się z 48-bitowego pola numeru rekordu w MFT oraz 16-bitowego *numeru sekwencji*; przy każdym ponownym użyciu danego rekordu (np. wskutek wcześniejszego skasowania jakiegoś pliku), numer sekwencji jest zwiększany o 1, co umożliwia utrzymanie spójności woluminu i likwidację wskaźników odwołujących się do skasowanych już plików;

Pliki systemowe (metapliki)

- ♦ wszystkie dane w systemie NTFS przechowywane są w formie plików; początkowe 16 rekordów plikowych w tablicy MFT zawiera informacje o plikach systemowych (tzw. metaplikach); zestawienie tych plików zawiera poniższa tablica;
- ♦ omawiana wcześniej *nadrzędna tablica plików* jest w rzeczywistości plikiem o nazwie \$Mft, posiadającym częściową kopię zapasową w pliku o nazwie \$MftMirr (umieszczonym na końcu partycji (woluminu)); dane na temat tego jak duża część MFT znajduje się w kopii zapasowej są dość rozbieżne; niektóre opisy podają informacje o czterech sektorach, a w przypadku Windows XP – w zależności od wielkości, cztery rekordy lub pierwszy klaster; jeszcze inne opisy podają wartość 16 lub 3 rekordy.

Nr	Nazwa pliku	Opis
0	\$MFT	nadrzędna tablica plików (spis wszystkich plików)
1	\$MFTMirr	kopia zapasowa fragmentu MFT
2	\$LogFile	dziennik transakcji
3	\$Volume	informacje o partycji (woluminie)
4	\$AttrDef	tabela nazw atrybutów, ich numerów i opisów
5	*)	katalog główny
6	\$Bitmap	mapa bitowa wszystkich klastrów dysku (wolny/zajęty)
7	\$Boot	sektor ładujący
8	\$BadClus	lista uszkodzonych klastrów
9	\$Secure	lista zabezpieczeń nałożonych na pliki na dysku
10	\$UpCase	tablica zamiany małych liter na duże (W standardzie Unicode)
11	\$Extend	rożne zastosowania

12		rezerwowane do użycia w przyszłości
13		
14		
15		

*) Różne źródła podają znak dolara, kropkę albo znak dolara z kropką

Dziennik transakcji

- ◆ kolejny rekord zawiera rejestr (log) operacji dyskowych — przechowywane są w nim informacje o wszystkich operacjach wykonywanych na plikach i katalogach (kopiowanie, kasowanie, przesunięcie do innego katalogu, itp.); prowadzenie tego rejestru umożliwia odtwarzalność danych;
- ◆ dziennik transakcji (kronikowanie – ang. *journaling*) jest prowadzony w celu zwiększeniu bezpieczeństwa danych dyskowych; gdy dowolny plik jest tworzony, modyfikowany lub usuwany, system NTFS dodaje odpowiedni rekord do dziennika transakcji (dziennika zmian); każdy rekord w dzienniku zmian zajmuje w przybliżeniu 80-100 bajtów; faktyczny zapis danych następuje po rejestracji w dzienniku; w przypadku niepowodzenia w zapisie (wskutek błędów transmisji lub uszkodzeń) transakcje są wycofywane z dysku;
- ◆ między innymi omawiany dziennik może być przydatny do tworzenia przyrostowych kopii zapasowych; używa się do tego celu odpowiednich funkcji API, które są udostępniane przez system.

Katalogi

- ◆ dane katalogowe przechowywane są w postaci B-drzewa, w którym wykorzystuje się 3 rodzaje atrybutów:
 1. indeks główny – korzeń drzewa reprezentującego katalog
 2. alokacje indeksu - przechowuje węzły drzewa katalogu
 3. mapa bitowa - kontrola wolnych miejsc w katalogu
- ◆ wpis katalogowy zawiera całą nazwę pliku i kopię atrybutu „informacje standardowe” (uzasadniane jest to względami wydajnościowymi); korzeń trzymany jest w „indeksie głównym”, wskaźniki do węzłów o wielkości 4 kiB przechowywane są w atrybucie „alokacje indeksu” (w postaci przedziałów bloków na dysku); reprezentacje plików w drzewie uporządkowane są leksykograficznie po nazwach plików;
- ◆ rekordy katalogów zawierają informacje indeksujące i podobnie jak w przypadku plików, atrybuty katalogów mogą się nie zmieścić w jednym rekordzie; duże katalogi są wtedy organizowane w postaci posortowanego drzewa zrównoważonego (B-drzewo) i posiadają rekordy ze wskaźnikami do zewnętrznych klastrów (buforów indeksowych), zawierających wpisy w folderze, które nie mieszczą się w MFT; przeszukiwanie drzewa zrównoważonego jest dużo szybsze niż listy, szczególnie przy dużej ilości elementów.

Podstawowe koncepcje operacji na plikach

- ◆ używane obecnie systemy operacyjne udostępniają znaczną liczbę funkcji usługowych wykonujących operacje na plikach; funkcje te oferują różne sposoby odczytu i zapisu plików, tworzenia nowych, kasowania istniejących i wiele innych;
- ◆ większość z tych funkcji wywodzi się z systemu Unix, a ich specyfikacje API ukształtowały się w latach siedemdziesiątych ubiegłego stulecia; charakterystyczną cechą omawianych funkcji jest jednolite traktowanie plików i urządzeń zewnętrznych komputera (zwłaszcza urządzeń znakowych);
- ◆ pod koniec lat osiemdziesiątych firma Microsoft opracowała nieco inne API, dostosowane do specyfiki aplikacji sterowanych zdarzeniami w systemie Windows; po wielu zmianach aktualnie używana wersja Windows API oznaczona jest skrótem Win32 API;
- ◆ zarówno w przypadku funkcji definiowanych w Unixie (Linuksie) lub w Windows, API definiowane jest na poziomie języka C; często kompilatory języków C/C++ działające w środowisku Windows udostępniają funkcje biblioteczne w stylu Unixa, ale nie jest to regułą i niekiedy zmienia się w kolejnych wersjach bibliotek
- ◆ w dalszym ciągu przedstawimy typowe funkcje API wykonujące operacje na plikach, skupiając uwagę na funkcjach wywodzących się z Unixa, a także na wybranych funkcjach Win32 API; podane opisy ograniczają się do omówienia najważniejszych, podstawowych operacji wykonywanych przez funkcje usługowe — wyczerpujący opis można znaleźć w dokumentacji systemu Linux czy w opisie Win32 API;

- ◆ proces przekazuje do systemu operacyjnego żądania wykonywania różnych działań związanych z plikiem poprzez wywoływanie odpowiednich podprogramów wchodzących w skład systemu operacyjnego; zlecenie operacji na pliku zgłoszone przez proces obejmuje:
 - *inicjalizację* czyli otwarcie pliku (w odniesieniu do pliku już istniejącego) albo utworzenie (nowego) pliku;
 - *właściwe operacje* (odczyt, zapis, . . .);
 - *deinicjalizację* czyli zamknięcie pliku;
- ◆ żądanie otwarcia lub utworzenia pliku skierowane do systemu operacyjnego zawiera nazwę pliku, jego położenie i inne parametry — jeśli żądanie to zostanie zaakceptowane przez system operacyjny, to przekazuje on do programu *uchwyt pliku* (ang. file handle), będący liczbą identyfikującą wskazany plik; w dalszych operacjach plik ten będzie identyfikowany przez zwrócony *uchwyt pliku* (a nie przez nazwę i położenie);
- ◆ czasami zamiast terminu *uchwyt pliku* używa się terminu *deskryptor pliku*, *identyfikator pliku* lub *dojście*;
- ◆ w systemie Unix (Linux) uchwyty o numerach 0, 1, 2 przypisane są na stałe do standardowych urządzeń znakowych — niżej wymienione urządzenia są dostępne bez konieczności jakichkolwiek przygotowań (nie potrzeba otwierać urządzenia):
 - 0 — standardowe urządzenie wejściowe,
 - 1 — standardowe urządzenie wyjściowe,
 - 2 — standardowe urządzenie do sygnalizacji błędu;
 zazwyczaj uchwyty 0, 1, 2 są w pewnym stopniu ze sobą związane.
- ◆ rozwój różnych systemów komputerowych, a zwłaszcza przetwarzania rozproszonego, powoduje że próbuje się szukać innych, bardziej efektywnych i niezawodnych metod obsługi plików:

- m.in. rozważa się systemy, w których nie będzie możliwa edycja plików — w przypadku konieczności zmiany danych zawartych w pliku program będzie generował cały plik o nowej (lub dotychczasowej) nazwie;
- można również zrezygnować ze stosowania uchwytów plików, co oznacza że jakakolwiek operacja na pliku będzie wymagała podania pełnej nazwy pliku i innych niezbędnych parametrów.

Tworzenie, otwieranie i zamykanie plików

- ◆ wiele systemów operacyjnych, udostępniających API w stylu Unixa, oferuje funkcję `open`, która używana jest do otwierania (i tworzenia) plików; w Win32 API analogiczną rolę pełni funkcja `CreateFile`, która omawiana będzie później; prototyp funkcji `open` na poziomie języka C ma postać:

```
int open (const char * path, int oflag, ...);
```

- ◆ funkcja `open` zwraca uchwyt (deskryptor) pliku, który jest wartością typu `int`; wartość `-1` oznacza, że plik nie został otwarty (zob. dalszy opis);
- ◆ parametr *path* określa nazwę pliku, która może być poprzedzona ścieżką dostępu; parametr ten może być podany jako wskaźnik (np. nazwa tablicy) albo jawnie, np.

```
fd = open ("d:\\OS\\przyklady" , ...);
```

- ◆ parametr *oflag* powinien zawierać dokładnie jeden z trzech niżej wymienionych parametrów, i ewentualnie inne parametry:

`O_RDONLY` — tylko odczyt,
`O_WRONLY` — tylko zapis,
`O_RDWR` — odczyt lub zapis,

- ◆ obok jednego z ww. parametrów można też podać następne, poprzedzone operatorem sumy bitowej `|` :

`O_CREAT` — tworzenie pliku (jeśli nie istnieje),
`O_APPEND` — operacja zapisu będzie wykonywana na końcu pliku;
`O_TRUNC` — zmniejszenie rozmiaru pliku do 0 (jeśli podano parametr `O_WRONLY` lub `O_RDWR`)

- ◆ w wewnętrznej reprezentacji kodu w języku C, a także w systemie Unix (Linux), znak nowego wiersza reprezentowany jest przez kod 10, natomiast w Windows (i wcześniej w DOS) używane są dwa znaki: 13, 10; w tej sytuacji odczytywanie pliku tekstowego zawierającego kody 13, 10 wymaga przekształcenia tych kodów na kod 10; pary (13, 10) mogą jednak występować w innych plikach (np. graficznych) i nie mogą być zastąpione przed kod 10; stąd w implementacji funkcji `open` w środowisku Windows zdefiniowany dwa tryby otwarcia:

<code>O_BINARY</code>	— otwarcie pliku w trybie binarnym,
<code>O_TEXT</code>	— otwarcie pliku w trybie tekstowym;

- ◆ zatem w środowisku Windows w trybie binarnym plik odczytywany jest lub zapisywany w oryginalnej postaci, natomiast w trybie tekstowym (który jest trybem domyślnym) pary znaków (13, 10) są zastępowane przy odczytywaniu przez kod 10, i odwrotnie przy zapisie kod 10 jest zastępowany przez parę (13, 10);
- ◆ jeśli drugi parametr funkcji `open` ma postać `O_CREAT`, to trzeba podać trzeci parametr: `S_IREAD`, `S_IWRITE`, albo `S_IREAD | S_IWRITE`;
- ◆ dostępna jest też odrębna funkcja do tworzenia plików:

```
int creat (const char * path, mode_t mode);
```

- ◆ wywołanie tej funkcji w postaci `creat (path, mode)`, jest równoważne wywołaniu funkcji `open`:

```
open (path, O_WRONLY | O_CREAT | O_TRUNC, mode);
```

- ◆ zamknięcie pliku realizuje funkcja

```
int close (int uchwyt);
```

- ◆ jeśli funkcja `open`, `create` lub `close` zwróci wartość `-1`, to oznacza, że operacja się nie powiodła — w tym przypadku ustawiana jest także zmienna globalna `errno`, która może przyjąć wartości:

<code>EACCES</code>	odmowa dostępu,
<code>EINVACC</code>	błędny kod dostępu,
<code>EMFILE</code>	zbyt dużo otwartych plików,
<code>ENOENT</code>	plik lub katalog nie istnieje;

- ◆ prototypy podanych tu funkcji znajdują się w pliku nagłówkowym `<unistd.h>` (Linux), `<io.h>`, zaś stałe zdefiniowane są w plikach `<fcntl.h>` i `<sys\stat.h>`.

Odczyt i zapis plików

- ♦ w systemach wywodzących się z Unixa program użytkowy posługuje się funkcją systemową `read`, aby przesłać dane z pliku do pamięci operacyjnej; analogicznie, funkcja systemowa `write` używana do przesyłania danych z pamięci operacyjnej do pliku; prototypy funkcji na poziomie języka C mają postać:

```
ssize_t read (int uchwyt, void * buf, size_t nbyte);  
ssize_t write (int uchwyt, void * buf, size_t nbyte);
```

gdzie:

uchwyt — uchwyt pliku,

buf — bufor w programie,

nbyte — liczba przesyłanych bajtów,

- ♦ funkcje `read` i `write` zwracają liczbę przesłanych bajtów; wartość 0 oznacza, że wcześniej cały plik został odczytany (koniec pliku), zaś wartość `-1` oznacza błąd operacji; jeśli wywołana funkcja `read` usiłuje przeczytać więcej bajtów, niż jest w pliku, to funkcja pobiera ich tyle, ile jest do przeczytania;
- ♦ w przypadku błędu ustawiana jest także zmienna globalna `errno`, która może przyjąć wartości:

<code>EACCES</code>	odmowa dostępu,
<code>EBADF</code>	błędny uchwyt pliku;
- ♦ funkcję `read` można zastosować do odczytu danych ze standardowego urządzenia wejściowego (`uchwyt = 0`);
- ♦ poniżej pokazano przykład programu, który wyświetla na ekranie zawartość pliku podanego w linii wywołania;

```
#include <stdio.h>
#include <io.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

void pokazPlik(char *sciezka)
{
    char bufor[2048];
    int fd, ile;

    if (0 > (fd = open(sciezka, O_RDONLY)))
    {
        perror("pokazPlik.open");
        return;
    }

    while(0 < (ile = read(fd, bufor,
                          sizeof(bufor))))
    {
        bufor[ile] = 0;
        printf("%s", bufor);
    }
    close(fd);
}

int main(int argc, char *argv[])
{
    int i;

    for(i=1; i < argc; i++)
    {
        pokazPlik(argv[i]);
    }
    return 0;
}
```

- ◆ kolejny przykład ilustruje wyświetlanie tekstu poprzez standardowe urządzenie wyjściowe (uchwyt = 1);

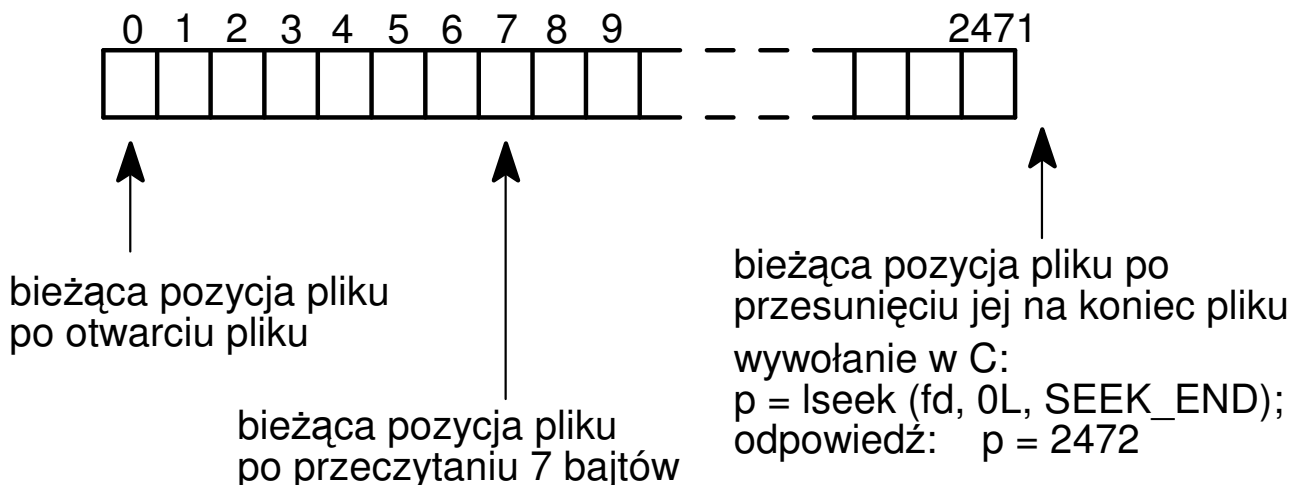
```
#include <stdio.h>
#include <io.h>

int main()
{
    char wiersz[]="Standardowe urządzenie \
wyjsciowe działa poprawnie\n";
    write (1, wiersz, sizeof(wiersz)-1);
    return 0;
}
```

Bieżąca pozycja pliku

- ♦ dostęp do plików ma charakter sekwencyjny, tj. każda operacja czytania lub pisania odnosi się do miejsca pliku tuż za pozycją osiągniętą przy poprzedniej operacji; zazwyczaj bajty w pliku są numerowane począwszy od zera;
- ♦ *bieżąca pozycja pliku* (ang. file pointer) ma charakter licznika odczytanych/zapisanych bajtów i określana jest zwykle za pomocą liczby 32-bitowej — każda operacja odczytu lub zapisu pliku będzie wykonana począwszy od bajtu wskazywanego przez bieżącą pozycję pliku; po odczycie lub zapisie bieżąca pozycja pliku jest przesuwana w taki sposób, że wskazuje kolejny bajt za ostatnio odczytanym lub zapisanym;
- ♦ system operacyjny umożliwia programom swobodny dostęp do pliku ze wskazaniem numeru bajtu jako punktu odniesienia; program użytkowy może ustawić bieżącą pozycję pliku na dowolnym bajcie i wykonywać operacje na danych począwszy od tej pozycji; kwestie te wyjaśnia poniższy przykład:

Z pliku zawierającego 2472 bajtów odczytano 7 bajtów.



- ♦ do przesuwania bieżącej pozycji pliku używana jest funkcja systemowa **lseek**, której prototyp ma postać:

```
off_t lseek(int uchwył, off_t offset, int skąd);
```

gdzie:

uchwył — uchwył pliku,

offset — wielkość przesunięcia (long),

skąd — punkt startowy: może przyjmować wartości:

SEEK_SET — początek pliku,

SEEK_CUR — bieżące położenie,

SEEK_END — koniec pliku

- ♦ funkcja zwraca bieżącą pozycję pliku po przesunięciu, albo wartość -1, jeśli wystąpił błąd (w tym przypadku ustawiana jest zmienna **errno**);
- ♦ omawiana funkcja pozwala więc na poruszanie się w pliku bez czytania lub pisania — można więc traktować pliki jak tablice;
- ♦ jeśli potrzebna jest tylko znajomość bieżącej pozycji pliku, to można korzystać z funkcji systemowej

```
p = tell (uchwył);
```

która zwraca bieżącą pozycję pliku (wartość typu long int).

Operacje plikowe w interfejsie Win32 API

- ◆ funkcje wykonujące operacje na plikach, zdefiniowane w interfejsie Win32 są na ogół bardziej skomplikowane niż ich odpowiedniki Unixowe; często w programach tworzonych dla systemu Windows używa się obu tych grup;
- ◆ rozpatrzmy funkcję Win32 API używaną do otwierania i tworzenia plików: **CreateFile**; ze względu na rozmiary opisu, ograniczymy się tylko do podania najważniejszych zastosowań tej funkcji; prototyp omawianej funkcji ma postać:

```
HANDLE CreateFile(
    LPCTSTR      lpFileName,
    DWORD        dwDesiredAccess,
    DWORD        dwShareMode,
    LPSECURITY_ATTRIBUTES lpSecurityAttributes,
    DWORD        dwCreationDisposition,
    DWORD        dwFlagsAndAttributes,
    HANDLE        hTemplateFile
);
```

- ◆ w systemie Windows nazwy zmiennych i parametrów funkcji tworzone są często zasad tzw. notacji węgierskiej; właściwa nazwa opisująca funkcje parametru poprzedzona jest dodatkowymi znakami, które określają typ; przykładowo, parametr **dwShareMode** zawiera na początku znaki **dw**, które oznaczają, że parametr typu **DWORD** (wartość 32-bitowa bez znaku);
- ◆ parametr **lpFileName** określa nazwę tworzonego lub otwieranego pliku (**lp** – long pointer – wskaźnik); w szczególnym przypadku funkcja **CreateFile** może być zastosowana do odczytywania zawartości dysku traktowanej jako ciąg sektorów — wówczas należy podać nazwę **\\.\PHYSICALDRIVE0**;

- ◆ parametr `dwDesiredAccess` określa przewidywany sposób dostępu do pliku i zazwyczaj określany jest przez stałe: `GENERIC_READ` lub `GENERIC_WRITE`; obie stałe można złożyć w postaci `GENERIC_READ | GENERIC_WRITE`, co oznacza, że plik będzie odczytywany i zapisywany;
- ◆ kolejny parametr `dwShareMode` określa sposób współdzielenia pliku przez inne procesy, np. wartość parametru `FILE_SHARE_READ` oznacza, że inne procesy mogą wykonywać operacje odczytu omawianego pliku;
- ◆ parametr `lpSecurityAttributes` w typowych zastosowaniach przyjmuje wartość `NULL`, co oznacza, że zwrócony uchwyt nie może być dziedziczony przez procesy potomne;
- ◆ parametr `dwCreationDisposition` opisuje sposób tworzenia i otwierania plików; zazwyczaj wartość tego parametru określona jest przez stałą, np. `OPEN_EXISTING` oznacza, że operacja dotyczy już istniejącego pliku (funkcja zwraca błąd jeśli plik nie istnieje), `CREATE_NEW` — funkcja zwraca błąd, jeśli plik już istnieje, `OPEN_ALWAYS` — plik zostanie otwarty lub utworzony (jeśli nie istniał), `TRUNCATE_EXISTING` — funkcja zwraca błąd, jeśli plik nie istnieje, ale jeśli plik istnieje, to kasuje jego zawartość (w tym przypadku należy wprowadzić wcześniej opisany parametr `GENERIC_WRITE`);
- ◆ parametr `dwFlagsAndAttributes` określa atrybuty pliku, np. `FILE_ATTRIBUTE_NORMAL` — zwykły plik, `FILE_ATTRIBUTE_HIDDEN` — plik ukryty, itp.;
- ◆ ostatni parametr `hTemplateFile` określa dodatkowe własności pliku i zazwyczaj przyjmuje wartość `NULL`;

- ◆ funkcja **CreateFile** zwraca uchwyt do otwartego lub utworzonego pliku w postaci wartości typu **HANDLE**; w przypadku błędu funkcja zwraca wartość **INVALID_HANDLE_VALUE**;
- ◆ funkcja **CloseHandle** używana jest do zamykania plików;
- ◆ inne funkcje wykonujące operacje na plikach w interfejsie Win32 API omówimy krótko na podstawie podanego niżej programu przykładowego; program ten zamienia znaki w pliku tekstowym: pierwszy z ostatnim, drugi z przedostatnim, itd.; nazwa pliku podana jest w linii wywołania programu;
- ◆ w programie używane są typy zmiennych w stylu Win32 API, które zwykle mają odpowiedniki w typach zdefiniowanych w języku ANSI C; w szczególności **DWORD** jest typem zmiennych 32-bitowych bez znaku, **CHAR** oznacza to samo co **char** w języku C, zaś **PCHAR** jest wskaźnikiem do wartości typu **CHAR**;

```
#include <windows.h>
```

```
#include <stdio.h>
```

```
int main (INT argc, PCHAR argv [ ])
```

```
{
```

```
    HANDLE uchwyt;
```

```
    DWORD i, rozmiar_pliku, liczba_bajtow;
```

```
    PCHAR wskObszar; CHAR tmp;
```

```
    if (argc != 2)
```

```
    {
```

```
        printf("\nNiewlasciwa liczba parametr. - koniec programu");
```

```
        return 2;
```

```
    }
```

```
    uchwyt = CreateFile (argv [1],
```

```
        GENERIC_READ | GENERIC_WRITE,
```

```
        FILE_SHARE_READ, NULL, OPEN_EXISTING,
```



```

    FILE_ATTRIBUTE_NORMAL, NULL);
if (uchwyt == INVALID_HANDLE_VALUE)
{ printf("\nBład otwarcia pliku - koniec programu"); return 3;}

```

```

// wyznaczanie rozmiaru pliku
rozmiar_pliku = GetFileSize (uchwyt, NULL);
if (rozmiar_pliku == INVALID_FILE_SIZE)
{
    printf("\nBledny rozmiar pliku - koniec programu");
    CloseHandle(uchwyt); return 8;
}

```

```

// przydzielanie obsz. pamięci globalnej przeznaczonej na plik
wskObszar = (PCHAR) GlobalAlloc (GMEM_FIXED,
                                rozmiar_pliku);
if (wskObszar == NULL)
{
    printf("\nBrak pamieci - koniec programu");
    CloseHandle(uchwyt); return 8;
}

```

```

ReadFile (uchwyt, (PVOID) wskObszar, rozmiar_pliku,
          &liczba_bajtow, NULL);
if (liczba_bajtow != rozmiar_pliku)
{
    printf("\nOdczyt.niewłaśc.liczbę bajtów - koniec programu");
    CloseHandle(uchwyt); return 5;
}

```

```

// zamiana znaków
for (i = 0; i < (rozmiar_pliku / 2); i++)
{
    tmp = * (wskObszar + i);
    * (wskObszar + i) = * (wskObszar + rozmiar_pliku - 1 - i);
    * (wskObszar + rozmiar_pliku - 1 - i) = tmp;
}

```

```

// przesunięcie pozycji bieżącej na początek pliku
SetFilePointer(uchwyt, 0, NULL, FILE_BEGIN);
WriteFile(uchwyt, wskObszar, rozmiar_pliku,
          &liczba_bajtow, NULL);

CloseHandle(uchwyt);
GlobalFree(wskObszar);
if (liczba_bajtow == rozmiar_pliku)
{
    printf("\nZamiana znakow przeprowadzona pomyslnie\n");
    return 0;
}
else
{
    printf("\nZamiana znaków przeprowadzona błędnie\n");
    return 9;
}
}

```

- ♦ rozmiar pliku wyznacza się za pomocą funkcji `GetFileSize` (`uchwyt, NULL`); funkcja ta zwraca rozmiar pliku w postaci liczby 64-bitowej; drugi parametr używany jest do wyznaczenia rozmiaru plików większych od 4GB — wówczas parametr ten zawiera wskaźnik do zmiennej, do której zostaną wpisane starsze 32 bity rozmiaru pliku.

Funkcje ReadFile i WriteFile

- ◆ do odczytu i zapisu plików w Win32 API zdefiniowano funkcje:

`ReadFile (hFile, pBuffer, dwToRead, &dwHaveRead, NULL);`

`WriteFile (hFile, pBuffer, dwToWrite, &dwHaveWritten, NULL);`

gdzie

`pBuffer` — adres obszaru (bufora) zawierającego dane;

`dwToRead/dwToWrite` — liczba odczytywanych/zapisywanych bajtów;

`dwHaveRead/dwHaveWritten` — zmienna, do której zostanie wpisana liczba odczytanych/zapisanych bajtów;

- ◆ czwarty parametr określa specjalny tryb operacji odczytu/zapisu i zazwyczaj przyjmuje wartość `NULL`;
- ◆ współczesne systemy operacyjne oferują także operacje odczytu i zapisu plików wykonywane w sposób asynchroniczny, co oznacza że funkcja odczytu lub zapisu kończy się od razu, bez oczekiwania na zakończenie właściwej operacji; zatem wykonanie funkcji nie oznacza, że żądany plik lub jego fragment został odczytany (lub zapisany); pozwala to na bardziej efektywne zakodowanie programu: w trakcie oczekiwania na wykonanie operacji plikowej można wykonywać inne operacje; w Win32 API operacje plikowe asynchroniczne wykonywane są przez funkcje `ReadFileEx` i `WriteFileEx`, natomiast w środowiskach Unixowych stosowane są funkcje `aio_read()` i `aio_write()`;

Komunikaty o błędach funkcji

- ♦ typowe wywołanie funkcji `read` ma postać:

```
k = read (uchwyt_pliku, adres_bufora, liczba_bajtow);
```

- ♦ wartość `k` reprezentuje liczbę przeczytanych bajtów, a może też sygnalizować błąd; wskazane jest testowanie wartości zwracanych przez funkcje `read` i `write`, np.

```
if (( n = read ( fd, bufor, liczba_b)) == -1)
{
    printf ("\nBłąd czytania pliku\n") ;
    exit (1);
};
```

- ♦ można też uzyskać dokładniejsze informacje o sygnalizowanym błędzie:

```
#include <stdio.h>
.....
extern int  errno, sys_nerr;
extern char * sys_errlist [ ];
.....
fprintf (stderr, "%s", sys_errlist[errno]);
.....
```

- ♦ informacje o błędach funkcji bibliotecznych wygodnie jest przekazywać za pomocą funkcji `perror`; funkcja ta kieruje znaki do standardowego urządzenia do sygnalizacji błędu (na ekran);

- ♦ funkcja `perror` wyświetla tekst podany jako argument funkcji, a następnie (po dwukropku) opis błędu (w jęz. angielskim); przykładowo, jeśli w poniższym fragmencie programu

```
int  uchwyty;  
uchwyty = open("zbiory.txt", O_RDONLY);  
if (uchwyty == -1) perror ("Błąd funkcji open");
```

próba otwarcia pliku `zbiory.txt` zakończy się niepomyślnie, to zostanie wyświetlony tekst:

Błąd funkcji open: No such file or directory

- ♦ zauważmy, że tekst po dwukropku został dopisany przez funkcję `perror` na podstawie wartości zmiennej globalnej `errno`.

Przeglądanie zawartości katalogów za pomocą funkcji `opendir`, `readdir` i `closedir`

- ◆ w programach wykonujących działania na plikach zachodzi często konieczność przeglądania (i ewentualnie wyświetlania) listy plików umieszczonych we wskazanym katalogu;
- ◆ w środowisku Unix (Linux) odczytywanie zawartości katalogów można wykonać za pomocą funkcji `opendir`, `readdir`, `rewinddir` i `closedir`; funkcje te są zgodne ze standardem POSIX (ang. Portable Operating System Interface), a ich prototypy podane są w pliku nagłówkowym `<dirent.h>`;

<code>opendir</code>	otwarcie strumienia danych katalogowych,
<code>readdir</code>	odczytanie ze strumienia danych katalogowych pojedynczej pozycji katalogu (kolejne wykonanie funkcji <code>readdir</code> spowoduje odczytanie kolejnej pozycji katalogu),
<code>rewinddir</code>	przesunięcie strumienia na pierwszą pozycję katalogu,
<code>closedir</code>	zamknięcie strumienia danych katalogowych;

- ◆ podane funkcje działają na strukturach `DIR` i `dirent`; funkcja `readdir` zwraca wskaźnik do struktury `dirent`, która może zawierać wiele pól, ale tylko pole

`char * d_name;`

zostało formalnie zdefiniowane w specyfikacji POSIX;

- ◆ pole to zawiera nazwę (np. pliku), zakończoną bajtem zerowym; przekazywane nazwy mogą stanowić: nazwy plików (także plików ukrytych i systemowych), nazwy katalogów (w tym katalogi `.` i `..`), etykiety dysku oraz nazwy urządzeń; funkcja `readdir` zwraca wskaźnik `NULL` w przypadku natrafienia na koniec katalogu, a także w przypadku błędu;

- ◆ technikę stosowania podanych funkcji wyjaśnia poniższy przykład

```
#include <stdio.h>
#include <sys/types.h>
#include <dirent.h>

int main ( )
{
    DIR * dirp;
    struct dirent * dp;

    printf("\n\n");
    dirp = opendir(".");
    /* odczytywanie bieżącego katalogu */
    while ((dp = readdir(dirp)) != NULL)
    {
        printf("\nPlik = %s", dp -> d_name);
    }
    /* od nowa */
    rewinddir(dirp);
    while ((dp = readdir(dirp)) != NULL)
    {
        printf("\nPlik = %s", dp -> d_name);
    }
    closedir (dirp);
    printf("\nTest programu\n");
    return 0;
}
```

- ◆ funkcja `readdir` nie podaje jednak informacji czy nazwa wskazuje na plik, czy na katalog czy też na urządzenie; informacje te można uzyskać za pomocą funkcji `stat`, której prototyp ma postać

```
int stat (const char * sciezka, struct stat * buf);
```

- ♦ funkcja ta, dla pliku określonego przez parametr **ściezka**, wpisuje informacje do struktury, której adres podaje drugi parametr; w przypadku błędu funkcja zwraca wartość **-1**;
- ♦ poniższy fragment (stanowiący rozszerzenie podanego wcześniej przykładu) ilustruje zastosowanie funkcji **stat**;

```
#include <sys\stat.h>
struct stat  statbuf;

/* odczytywanie bieżącego katalogu */
while ((dp = readdir(dirp)) != NULL)
{
    printf("\nPlik = %s", dp -> d_name);
    stat (dp -> d_name, &statbuf);
    if (statbuf.st_mode & S_IFDIR) printf(" - jest to katalog!");
    else printf (" - rozmiar pliku = %ld", statbuf.st_size);
}
```

- ♦ używana jest także funkcja **fstat**, która pozwala uzyskać analogiczne informacje w odniesieniu do plików aktualnie otwartych (zamiast nazwy pliku podaje się uchwyt pliku);
- ♦ należy zwrócić uwagę, że nazwy plików uzyskiwane za pomocą funkcji **readdir** trzeba poprzedzić ścieżką dostępu, i dopiero w tej postaci nazwy można zastosować jako argumenty funkcji **stat**; ścieżkę dostępu można pominąć w przypadku katalogu bieżącego.

Przeglądanie zawartości katalogów za pomocą funkcji **FindFirstFile** i **FindNextFile**

- ◆ w interfejsie Win32 API dostępne są funkcje **FindFirstFile** i **FindNextFile**, które mogą być wygodniejsze niż opisane wcześniej funkcje **opendir**, **readdir**, **closedir**;
- ◆ w wyniku wywoływania podanych funkcji uzyskuje się informacje o poszczególnych plikach podanego katalogu w postaci struktury o następujących polach:

```
typedef struct _WIN32_FIND_DATA { // wfd
    DWORD dwFileAttributes;
    FILETIME ftCreationTime;
    FILETIME ftLastAccessTime;
    FILETIME ftLastWriteTime;
    DWORD nFileSizeHigh;
    DWORD nFileSizeLow;
    DWORD dwReserved0;
    DWORD dwReserved1;
    TCHAR cFileName[ MAX_PATH ];
    TCHAR cAlternateFileName[ 14 ];
} WIN32_FIND_DATA;
```

- ◆ podany niżej program wyświetla nazwy i rozmiary plików znajdujących się w katalogu wskazanym w linii wywołania programu;

```
#include <windows.h>
#include <string.h>
#include <stdio.h>
```

```
int main (INT argc, PCHAR argv [ ])
{
    CHAR katalog[512]; BOOL odpb; HANDLE h2;
```

```
WIN32_FIND_DATA pliki_znal;
```

```
if (argc != 2)
{
    printf("\nBrak nazwy katalogu - koniec programu");
    return (5);
}
```

```
strcpy(katalog, argv[1]); strcat(katalog, "\\*.");
```

```
h2 = FindFirstFile ((LPCTSTR) katalog, &pliki_znal);
if (h2 == INVALID_HANDLE_VALUE)
{ printf("\nBrak plików"); return 2; }
```

```
do
{
    if (!(pliki_znal.dwFileAttributes &
        FILE_ATTRIBUTE_DIRECTORY))
    {
        printf("\n%s    rozmiar = %d", pliki_znal.cFileName,
            pliki_znal.nFileSizeLow);
    }
    odpb = FindNextFile(h2, &pliki_znal);
} while (odpb);
```

```
FindClose(h2);
return 0;
```

```
}
```

Inne operacje na plikach

♦ funkcja

`int chmod (const char * sciezka, mode_t mode);` (Linux)

`int chmod (const char * sciezka, int mode);` (Windows)

wywodzi się z Unixa, ale udostępniana jest także przez biblioteki kompilatorów działające w środowisku Windows; funkcja ta zmienia uprawnienia do wykonywania operacji na pliku wskazanym przez parametr *sciezka*;

- ♦ ze względu na odmienne sposoby określania uprawnień w obu systemach, drugi parametr *mode* zależy od systemu; w przypadku Windows może on przyjmować wartości: `S_IWRITE` (dozwolony tylko zapis), `S_IREAD` (dozwolony tylko odczyt), `S_IWRITE | S_IREAD` (dozwolony odczyt i zapis); jeśli dozwolony jest zapis, to jednocześnie dozwolony jest odczyt, wobec czego parametry `S_IWRITE` i `S_IWRITE | S_IREAD` mają taki sam skutek;

- ♦ podobnie działa funkcja `fchmod`, w której zamiast nazwy pliku podaje się uchwyt pliku; funkcja ta nie jest dostępna w środowisku Windows;

♦ zmiana nazwy pliku

`rename (stara nazwa, nowa nazwa);`

w przypadku poprawnej realizacji funkcja zwraca wartość 0, w przeciwnym razie `-1`; przykład:

```
char nazwa_pliku [80];
```

```
— — — — — — — — — — — — — — — —
```

```
strcpy (nazwa_pliku, "nowy_sch.rys");
```

```
rename ("schemat.rys", nazwa_pliku);
```

- ♦ kasowanie pliku można wykonać za pomocą funkcji
`unlink (nazwa pliku);`

- ◆ zmianę katalogu można wykonać za pomocą funkcji systemowej
chdir (*nazwa katalogu*);
- ◆ tworzenie i kasowanie katalogu
mkdir (*nazwa katalogu*);
rmdir (*nazwa katalogu*);

Koncepcja strumieni w języku C

- ◆ niektóre programy są przystosowane do pobierania danych z różnych źródeł — program jednak powinien wykonywać działania na danych w taki sam sposób, niezależnie od źródła z którego pochodzą; zatem nadchodzące dane powinny być, za pomocą funkcji bibliotecznych właściwych dla poszczególnych źródeł, przekształcone do jednolitej postaci — uzyskane w ten sposób dane (jednolite) tworzą *strumień wejściowy*;
- ◆ analogicznie, wyniki tworzone przez program powinny mieć jednolitą postać, i dopiero w trakcie przekształcania na postać specyficzną dla danego urządzenia powinny przyjmować cechy właściwe dla tego urządzenia (to realizują funkcje biblioteczne); oznacza to, że program powinien kierować wyniki do *strumienia wyjściowego*;
- ◆ *strumień* jest źródłem lub celem danych i może być związany z dyskiem lub innym urządzeniem zewnętrznym;
- ◆ biblioteka standardowa obsługuje *strumienie tekstowe i binarne*:
 - *strumień tekstowy* w języku C jest ciągiem wierszy (każdy wiersz zawiera pewną liczbę znaków i kończy się znakiem '\n');
 - *strumień binarny* jest ciągiem nie przetworzonych bajtów;
- ◆ strumienie tekstowe i binarne w systemie Unix są identyczne;
- ◆ w środowiskach niektórych programów strumienie muszą być przekształcone do postaci zmodyfikowanej (zastąpienie '\n' przez znaki powrotu karetki i nowego wiersza);
- ◆ strumień wiąże się z plikiem lub urządzeniem poprzez operację otwarcia; operacja zamknięcia oznacza przerwanie tego połączenia.

Operacje plikowe w języku C wykonywane na strumieniach

- ◆ omówione wcześniej funkcje systemowe stanowią zestaw pozwalający na efektywne wykonywanie różnych operacji na plikach; jednak pewne funkcje systemowe są specyficzne dla konkretnego środowiska, a zatem *nieprzenośne*, co utrudnia przenoszenie programów z jednego środowiska do innego;
- ◆ wyrazem dążenia do zapewnienia przenośności programów stało się opracowanie w latach osiemdziesiątych standardu języka C znanego jako ANSI C — m.in. w standardzie tym zostały zdefiniowane funkcje działające na strumieniach, nie są natomiast zdefiniowane funkcje `open`, `read`, `write`, . . . ;
- ◆ operacja otwarcia wiąże plik ze strumieniem — w tym celu powoduje utworzenie obszaru sterowania o strukturze `FILE`; obszar sterowania zawiera różne pola niezbędne do obsługi strumienia — wskaźnik do tego obszaru, nazywany *wskaźnikiem pliku* zwracany jest przez funkcję `fopen`; czasami używa się zamiennie określeń *wskaźnik do pliku* i *strumień*;
- ◆ otwarcie (utworzenie) pliku

```
FILE * fp ;
-----
fp = fopen (nazwa, tryb);
```

gdzie:

nazwa — nazwa pliku podana jawnie albo w postaci wskaźnika do łańcucha znaków;

tryb — rodzaj operacji wykonywanej na pliku wg poniższej tablicy:

"r"	otwarcie pliku tekstowego dla czytania; jeśli wskazany plik nie został znaleziony, to funkcja <code>fopen</code> zwróci błąd;
"w"	otwarcie pliku tekstowego do pisania; skasowanie poprzedniej zawartości pliku jeśli istnieje;
"a"	dopisywanie; otwarcie lub utworzenie pliku tekstowego do dopisywania na końcu pliku;
"r+"	otwarcie pliku tekstowego do czytania i pisania; jeśli wskazany plik nie został znaleziony, to funkcja <code>fopen</code> zwróci błąd;
"w+"	otwarcie pliku tekstowego do czytania i pisania; skasowanie poprzedniej zawartości jeśli istnieje;
"a+"	otwarcie lub utworzenie pliku tekstowego dla czytania i dopisywania;

- ◆ wymienione operacje wykonywane są w trybie tekstowym — w trybie tym sekwencja kodów CR, LF (0x0D, 0x0A) odczytana z pliku jest zastępowana przez kod LF, i analogicznie kod LF zapisywany do pliku jest zastępowany przez CR, LF;
- ◆ dodatkowa litera `b`, występująca w podanych wyżej trybach oznacza, że operacja zostanie wykonana w trybie binarnym, np. `"w+b"`; tryb tekstowy jest trybem domyślnym: pominięcie litery `b` lub wpisanie litery `t` oznacza, że operacja zostanie wykonana w trybie tekstowym;
- ◆ gdy program rozpoczyna działanie, otwarte są trzy strumienie:
 - `stdin` (standardowe wejście),
 - `stdout` (standardowe wyjście),
 - `stderr` (standardowe wyjście błędów);
- ◆ różnice między funkcjami `open` i `fopen` :
 - funkcja systemowa `open` otwiera plik dla operacji czytania lub pisania; plik identyfikowany jest przez uchwyt;
 - funkcja `fopen` otwiera strumień, tzn. otwiera plik wskazany przez nazwę pliku i wiąże go ze strumieniem; strumień identyfikowany jest przez wskaźnik zwracany przez funkcję —

praktycznie: plik identyfikowany jest przez wskaźnik, który może być argumentem wielu różnych funkcji działających na pliku;

♦ przesuwanie bieżącej pozycji pliku

`fseek (fp, offset, skąd);`

gdzie:

`fp` — strumień (wskaźnik pliku);

`offset` — wielkość przesunięcia (long),

`skąd` — parametr ten określa punkt startowy i może przyjmować wartości (int):

`SEEK_SET` — początek pliku,

`SEEK_CUR` — bieżące położenie,

`SEEK_END` — koniec pliku

funkcja `fseek` zwraca wartość 0 w przypadku realizacji poprawnej, inaczej zwraca wartość różną od zera;

♦ przesuwanie bieżącej pozycji pliku na początek pliku

`rewind (fp);`

gdzie:

`fp` — strumień (wskaźnik pliku);

funkcja `rewind` nie zwraca żadnej wartości;

♦ jeśli potrzebna jest tylko znajomość bieżącej pozycji pliku, to można korzystać z funkcji

`p = ftell (fp);`

która zwraca bieżącą pozycję pliku (long);

- ◆ uchwyt pliku umieszczony jest w obszarze związanym z plikiem (pole `fd`) — można go uzyskać za pomocą jednego z niżej podanych sposobów:

```
k = (*fp).fd ;
k = fp -> fd ;
k = fileno (fp) ;
```

- ◆ makrodefinicja `fileno` jest zdefiniowana następująco:

```
#define      fileno(f)      ((f)->fd)
```

- ◆ zamknięcie pliku

```
fclose (fp);
```

funkcja `fclose` powoduje zerwanie połączenia między wskaźnikiem pliku a nazwą zewnętrzną, ustalonego przez funkcję `fopen` ; zamknięcie pliku powoduje także wykonanie szeregu czynności dodatkowych jak np. zlikwidowanie uprzednio przydzielonych buforów.

Wprowadzanie i wyprowadzanie danych znakowych w języku C

Funkcje pobierania znaków i łańcuchów

	Z klawiatury	Z "stdin"	Z pliku określ. przez FILE
Plik nagłówek.	conio.h	stdio.h	stdio.h
Znaki	getch – bez echa getche – z echem	getchar – makro fgetchar – funkcj.	getc – makro fgetc – funkcja
Łańcuchy	cgets	gets	fgets
Bloki danych			fread
Formatowane	cscanf	scanf	fscanf
Oddawanie znaku	ungetch		ungetc

Funkcje wysyłania znaków i łańcuchów

	Do okna tekstowego	Do "stdout"	Do pliku określ. przez FILE
Plik nagłówek.	conio.h	stdio.h	stdio.h
Znaki	putch	putchar – makro fputchar – funkcja	putc – makro fputc – funkcja
Łańcuchy	cputs	puts	fputs
Bloki danych			fwrite
Formatowane	cprintf	printf	fprintf

/* Wyświetlanie wierszy pliku TEKST.BAT, w których występuje łańcuch '.EXE'; wersja I — wykorzystuje operacje plikowe na poziomie podstawowym

*/

#include <stdio.h>

#include <string.h>

#include <io.h>

#include <fcntl.h>

int main ()

{

char buf[1000]; int ident,k; char *p, *q;

printf("\n");

ident = open ("tekst.bat", O_RDONLY);

if (ident == -1) return (1);

k = read (ident, buf, sizeof (buf)-1); close (ident);

buf [k] = '\0'; p = buf;

while (p < buf + k)

{

/* funkcja *strchr* poszukuje w łańcuchu p znaku o kodzie 10 i zwraca wskaźnik znalezionej miejsca*/

q = strchr (p, 10);

q = '\0'; / zastąpienie kodu 10 przez 0 */

strupr (p); /* zamiana na wielkie litery */

/* funkcja *strstr* poszukuje ciągu znaków EXE w łańcuchu p — zwraca NULL jeśli nie znajdzie */

if (strstr (p, ".EXE") != NULL) printf ("\n%s",p);

p = q + 1; /* przejście do następnego wiersza */

}

return(0);

}

```
/* Wyświetlanie wierszy pliku TEKST.BAT, w których występuje
   łańcuch '.EXE'; wersja II — wykorzystuje operacje plikowe na
   strumieniach
*/
```

```
#include <stdio.h>
#include <string.h>
```

```
int main ( )
{
    int i; char lanc [128];
    FILE *plik;

    printf ("\n");
    plik = fopen ("tekst.bat", "r");
    if (plik == NULL) return (1);

    /* funkcja fgets pobiera znaki z pliku wskazanego przez
       strumień plik i umieszcza je w tablicy lanc — operacja
       zostaje przerwana jeśli natrafiono na znak nowej linii
       lub pobrano (tu) 127 znaków */
    while (fgets (lanc, 128, plik) != NULL)
    {
       strupr (lanc); /* zamiana na wielkie litery */

        /* funkcja strstr poszukuje ciągu znaków EXE
           w łańcuchu p — zwraca NULL jeśli nie znajdzie */
        if (strstr (lanc, ".EXE") != NULL) printf ("%s", lanc);
    }
    fclose (plik);
    return (0);
}
```

Odzworowanie pliku w pamięci operacyjnej

- ♦ w systemie Windows istnieje możliwość utworzenia tablicy bajtowej tożsamej z zawartością pliku — wszelkie działania wykonywane na tej tablicy są w istocie działaniami na pliku; ilustruje to poniższy przykład, w którym wywoływane są funkcje Win32 API:

```
HANDLE    hPlik, hMap;  
char * p;  
DWORD rozmiar;
```

```
hPlik = CreateFile ("d:\\Builder\\teksty1.txt",  
                  GENERIC_READ | GENERIC_WRITE,  
                  0, NULL, OPEN_ALWAYS,  
                  FILE_ATTRIBUTE_NORMAL, 0);  
// trzeci parametr 0 oznacza, że nikt inny nie będzie mógł  
// korzystać z pliku, FILE_SHARE_READ lub  
// FILE_SHARE_WRITE oznacza dostęp do odczytu lub zapisu  
// ostatni parametr 0 występuje też jako NULL
```

```
// wyznaczanie rozmiaru pliku (< 4GB)  
rozmiar = GetFileSize (hPlik, NULL);
```

```
// tworzenie nowego obiektu typu „plik odwzorowania  
// (mapowania) pamięci” – w tym momencie nie jest jeszcze  
// rezerwowana pamięć  
hMap = CreateFileMapping (hPlik, NULL,  
                          PAGE_READWRITE, 0, 0, NULL);
```

```
/* trzeci parametr funkcji CreateFileMapping określa tryb  
współdzielenia z innymi procesami (może przyjmować  
wartości: PAGE_READONLY, PAGE_WRITECOPY,  
PAGE_READWRITE); parametr powinien być zgodny z trybem
```

współdzielenia podanym jako parametr funkcji CreateFile */

```
// tworzenie odwzorowania pliku w przestrzeni adresowej
// pamięci
```

```
p = (char *) MapViewOfFile (hMap,
                           FILE_MAP_ALL_ACCESS, 0, 0, 0);
```

/* Uwagi dot. funkcji MapViewOfFile

1. funkcja zwraca wskaźnik do obszaru pamięci pokrywającego się z zawartością pliku (lub jego części);
2. początek odwzorowywanego obszaru wskazany jest przez 64-bitowy offset wskazany przez trzeci i czwarty parametr (tu: 0,0 — high, low); parametry te muszą razem wskazywać offset, którego wartość musi być wielokrotnością parametru ziarnistości przydziału pamięci (zwykle 64KB) — inaczej funkcja zwraca błąd;
3. ostatni parametr podaje wielkość odwzorowywanego obszaru — jeśli parametr ten zawiera 0, to cały plik jest odwzorowywany
4. podzielenie całej operacji na dwa etapy realizowane przez funkcje CreateFileMapping i MapViewOfFile ma na wprowadzenie możliwości odwzorowania w przestrzeni adresowej procesu tylko części pliku — istnieją pliki większe niż 2 GB, których w całości nie dałoby się odwzorować
5. w podanym przykładzie pominięto obsługę błędów

*/

```
// zapis znaków # do pliku
```

```
*(p+10) = '#';
```

```
*(p+40) = '#';
```

```
*(p+rozmiar-1) = '\0';
```

```
// jeśli chcemy, żeby każda zmiana zawartości obszaru była od
// razu odwzorowywana na dysku, to stosuje się funkcję
// FlushViewOfFile(pFileData,dwViewSize);
```

// to wywołanie zamykania odwzorowania pliku w pamięci
UnmapViewOfFile(p);

CloseHandle(hMap);
CloseHandle(hPlik);

- ◆ powyższa technika może być także zastosowana do przesyłania danych między procesami wykonywanymi w systemie Windows.

Przykład: wyznaczanie ziarnistości przydziału pamięci.

```
/*   Wyznaczanie ziarnistości przydziału pamięci
    w systemu Windows
*/

#include <windows.h>
#include <stdio.h>

int main ( )
{
    SYSTEM_INFO s2;
    GetSystemInfo (&s2);
    printf("\nZiarnistość przydzielania pamięci = %u",
          s2.dwAllocationGranularity);
    return 0;
}
```